

OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 2

CODES

```
//1. Write a program in which a class named student has member variables name, roll_no,
//semester and section. Create 4 objects of this class to store data of 4 different students,
//now display data of only those students who belong to section A.
#include <iostream>
using namespace std;

class student{
public:
    string name;
    int rollno;
    int semester;
    string section;
// CONSTRUCTOR:
student(string n, int r,int sem , string sec){
    name = n;
    rollno = r;
    semester = sem;
    section = sec;
}

// DISPLAY FUNCTION:
void display(){
    cout<< name << " " << rollno << " " << semester << " " << section << endl;
}
};

int main(){
// OBJECTS:
    student s1("abdul", 190 , 2 , "D");
    student s2("ali",170 , 2 , "A");
    student s3("anas", 160 , 3 , "b");
    student s4("akber", 180 , 4 , "A");

    if (s1.section == "A") s1.display();
    if (s2.section == "A") s2.display();
    if (s3.section == "A") s3.display();
    if (s4.section == "A") s4.display();
    return 0;
}
```

OUTPUT:

```
ali 170 2 A
akber 180 4 A
```

```

1 //2. You are a programmer for the ABC Bank assigned to develop a class that models the basic
2 //workings of a bank account. The class should perform the following tasks:
3 //o Save the account balance.
4 //o Save the number of transactions performed on the account.
5 //o Allow deposits to be made to the account.
6 //o Allow with drawls to be taken from the account.
7 //o Report the current account balance at any time.
8 //o Report the current number of transactions at any time.
9 //Menu
10 //1. Display the account balance
11 //2. Display the number of transactions
12 //3. Display interest earned for this period
13 //4. Make a deposit
14 //5. Make a withdrawal
15 //6. Exit the program
16 #include <iostream>
17 using namespace std;
18
19 class BankAccount {
20 public:
21     int balance;
22     int transactions;
23
24     BankAccount() {
25         balance = 0;
26         transactions = 0;
27     }
28
29     void deposit(int amount) {
30         balance += amount;
31         transactions++;
32     }

```

```

29 void deposit(int amount) {
30     balance += amount;
31     transactions++;
32 }
33
34 void withdraw(int amount) {
35     balance -= amount;
36     transactions++;
37 }
38
39 void showBalance() {
40     cout << "Balance: " << balance << endl;
41 }
42
43 void showTransactions() {
44     cout << "Transactions: " << transactions << endl;
45 }
46 };
47
48 int main() {
49     BankAccount acc;
50
51     acc.deposit(5000);
52     acc.deposit(300);
53     acc.withdraw(4000);
54
55     acc.showBalance();
56     acc.showTransactions();
57
58     return 0;
59 }

```

OUTPUT:

```

Balance: 1300
Transactions: 3

```

```

1 //Q3. Create a class called Employee that includes three pieces of information as data
2 //members—a first name (type char* (DMA)), a last name (type string) and a monthly salary
3 //(type int). Your class should have a setter function that initializes the three data members.
4 //Provide a getter function for each data member. If the monthly salary is not positive, set it
5 //to 0. Write a test program that demonstrates class Employee's capabilities. Create two
6 //Employee objects and display each object's yearly salary. Then give each Employee a 10
7 //percent raise and display each Employee's yearly salary again. Identify and add any other
8 //related functions to achieve the said goal.
9 #include<iostream>
10 #include<string>
11 #include<cstring>
12
13 using namespace std;
14
15 class employee{
16     private:
17         char* firstname;
18         string lastname;
19         int monthlySalary;
20
21     public:
22         employee(){
23             firstname = NULL;
24             lastname = "";
25             monthlySalary = 0;
26         }
27
28         void setemployee(const char* fname, string lname, int salary){
29             firstname = new char[strlen(fname) + 1];
30             strcpy(firstname, fname);
31
32             lastname = lname;
33
34             if(salary > 0){
35                 monthlySalary = salary;
36             }
37             else{
38                 monthlySalary = 0;
39             }
40         }
41
42         void display(){
43             cout << "Employee's Name: " << firstname << " " << lastname << endl;
44             cout << "Employee's Monthly Salary: " << monthlySalary << endl;
45             cout << "Employee's Yearly Salary: " << 12 * monthlySalary << endl;
46         }
47
48         void raiseSalary(int percent){
49             int raise = (percent * monthlySalary) / 100;
50             monthlySalary += raise;
51         }
52
53         void getEmployeeDetails(){
54             char* fname;
55             string lname;
56             int salary;
57
58             cout << "Enter first name: ";
59             fname = new char[50];
60             getLine(fname);
61
62             cout << "Enter last name: ";
63             getLine(lname);
64
65             cout << "Enter monthly salary: ";
66             salary = getInteger();
67
68             setemployee(fname, lname, salary);
69         }
70
71         void getLine(char* str){
72             int i = 0;
73             while (i < 50 && (getchar() != '\n'))
74                 str[i++] = getchar();
75             str[i] = '\0';
76         }
77
78         int getInteger(){
79             int num = 0;
80             while (getchar() != '\n')
81                 num = num * 10 + (getchar() - '0');
82             return num;
83         }
84 };
85
86 int main(){
87     employee emp1, emp2;
88     emp1.getEmployeeDetails();
89     emp2.getEmployeeDetails();
90     emp1.display();
91     emp2.display();
92     emp1.raiseSalary(10);
93     emp2.raiseSalary(10);
94     emp1.display();
95     emp2.display();
96     return 0;
97 }

```

```

37     else{
38         monthlySalary = 0;
39     }
40 }
41
42 char* getFirstname(){
43     return firstname;
44 }
45
46 string getLastName(){
47     return lastname;
48 }
49
50 int getYearlysalary(){
51     return monthlySalary * 12;
52 }
53
54 void Giveraise(){
55     monthlySalary += monthlySalary * 0.10;
56 }
57
58 int getMonthlysalarly(){
59     return monthlySalary;
60 }
61
62 ~employee(){
63     delete[] firstname;
64 }
65 };
66
67 int main(){
68     employee e1, e2;
69
70     e1.setemployee("Abdul", "Raheem", 50000);
71     e2.setemployee("Ali", "khan", 60000);
72
73     cout<<"\n===Yearly salary before raise===\n"<<endl;
74     cout<<e1.getFirstname()<<" "<<e1.getLastName()<<": "<<e1.getYearlysalary()<<endl;
75     cout<<e2.getFirstname()<<" "<<e2.getLastName()<<": "<<e2.getYearlysalary()<<endl;
76
77     e1.Giveraise();
78     e2.Giveraise();
79
80     cout<<"\n===Yearly salary after raise===\n"<<endl;
81     cout<<e1.getFirstname()<<" "<<e1.getLastName()<<": "<<e1.getYearlysalary()<<endl;
82     cout<<e2.getFirstname()<<" "<<e2.getLastName()<<": "<<e2.getYearlysalary()<<endl;
83
84     return 0;
85 }

```

OUTPUT:

```

===Yearly salary before raise===

Abdul Raheem: 600000
Ali khan: 720000

===Yearly salary after raise===

Abdul Raheem: 660000
Ali khan: 792000

```

```

1 //Q4. Write C++ code to represent a hitting game by using OOP concept. The details are as
2 //follows: This game is being played between two teams (i.e. your team and the enemy
3 //team). The total number of players in your team is randomly generated and stored
4 //accordingly. The function generates a pair of numbers and matches each pair. If the
5 //numbers get matched, the following message is displayed: "Enemy got hit by your team!"
6 //Otherwise, the following message is displayed: "You got hit by the enemy team!" The
7 //number of hits should be equal to the number of players in your team. The program should
8 //tell the final result of your team by counting the hits of both the teams. Consider the
9 //following sample output:
10 #include <iostream>
11 #include <cstdlib>
12 #include <ctime>
13 using namespace std;
14
15 class Game {
16 public:
17     int players, wins, losses;
18
19     Game() : players(rand() % 10 + 1), wins(0), losses(0) {}
20
21     void hit(void) {
22         int num1 = rand() % 5 + 1;
23         int num2 = rand() % 5 + 1;
24         cout << "Pair of Numbers:" << endl;
25         cout << "Number 1: " << num1 << endl;
26         cout << "Number 2: " << num2 << endl;
27         if (num1 == num2) {
28             cout << "Enemy got hit by your team!" << endl;
29             wins++;
30         } else {
31             cout << "You got hit by the enemy team!" << endl;
32             losses++;
33         }
34     }
35
36     void displayResult(void) {
37         cout << "Game Over! ";
38         if (wins > losses)
39             cout << "You won" << endl;
40         else if (losses > wins)
41             cout << "You lost" << endl;
42         else
43             cout << "The game is a draw" << endl;
44     }
45 };
46
47 int main(void) {
48     srand(time(0));
49     Game team;
50     cout << "Number of players in the team: " << team.players << endl;
51     for (int i = 0; i < team.players; i++)
52         team.hit();
53     team.displayResult();
54     return 0;
55 }

```

OUTPUT:

```
Number of players in the team: 1  
Pair of Numbers:  
Number 1: 3  
Number 2: 5  
You got hit by the enemy team!  
Game Over! You lost
```